

Google Cloud Platform

Deep Dive Tutorial

Managing Deployments Using Kubernetes Engine

&

Cloud Monitoring: Qwik Start

A Comprehensive Guide with Labs, Examples, and Hands-On Instructions

Version 1.0 | Last Updated: September 2026

Table of Contents

1. Introduction
2. Part 1: Managing Deployments Using Kubernetes Engine
 - 2.1 Understanding Kubernetes Fundamentals
 - 2.2 Kubernetes on Google Cloud Platform
 - 2.3 Lab 1: Creating and Managing GKE Clusters
 - 2.4 Lab 2: Deploying Applications
 - 2.5 Lab 3: Managing Deployments
3. Part 2: Cloud Monitoring
 - 3.1 Cloud Monitoring Overview
 - 3.2 Lab 4: Setting Up Cloud Monitoring
 - 3.3 Lab 5: Creating Dashboards
 - 3.4 Lab 6: Alerts and Policies
4. Advanced Topics
5. Troubleshooting Guide
6. Best Practices and Summary

1. Introduction

Welcome to this comprehensive tutorial on managing deployments using Google Kubernetes Engine (GKE) and monitoring your applications with Google Cloud Monitoring. This guide is designed for participants who want to gain hands-on experience with containerized applications and operational excellence in Google Cloud Platform.

Learning Objectives

By completing this tutorial, you will be able to:

- Understand Kubernetes architecture and concepts
- Create and manage Google Kubernetes Engine clusters
- Deploy containerized applications to GKE
- Manage and update deployments using rolling updates and blue-green deployments
- Set up comprehensive monitoring using Google Cloud Monitoring
- Create custom dashboards and metrics
- Implement alerting policies and notifications

Prerequisites

Before starting this tutorial, ensure you have:

An active Google Cloud Platform account with billing enabled
The Google Cloud SDK installed on your local machine
Basic understanding of containerization and Docker
Familiarity with Linux command line and bash
Basic knowledge of YAML syntax

About This Tutorial

This tutorial is structured in two main parts. The first part focuses on Kubernetes and GKE, progressing from fundamental concepts to advanced deployment strategies. The second part builds on your deployed applications by implementing comprehensive monitoring and alerting. Each section includes hands-on labs where you'll work directly with GCP resources.

2. Managing Deployments Using Kubernetes Engine

2.1 Understanding Kubernetes Fundamentals

What is Kubernetes?

Kubernetes (often abbreviated as K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. It provides a framework to manage distributed systems reliably, with mechanisms for deployment, maintenance, and scaling of applications.

Core Kubernetes Concepts

Pod: The smallest deployable unit in Kubernetes. A pod can contain one or more containers (usually Docker containers) that share network namespace. Think of it as a lightweight wrapper around a container.

Service: An abstract way to expose an application running on a set of pods as a network service. Services provide load balancing and stable endpoints for accessing your pods.

Deployment: A declarative update mechanism for Pods and ReplicaSets. Deployments describe the desired state of your application, and the Kubernetes controller continuously works to achieve and maintain that state.

ReplicaSet: Ensures that a specified number of pod replicas are running at any given time. Deployments manage ReplicaSets to provide declarative updates.

Namespace: Virtual clusters backed by the same physical cluster. This allows multiple teams or projects to share a cluster while maintaining isolation.

ConfigMap and Secret: Used to store configuration data and sensitive information respectively, which can be injected into pods at runtime.

2.2 Kubernetes on Google Cloud Platform

Google Kubernetes Engine (GKE)

Google Kubernetes Engine is a fully managed Kubernetes service in Google Cloud Platform. GKE takes care of the underlying infrastructure, allowing you to focus on deploying and managing your applications. Key benefits include:

Fully managed by Google - automatic updates, patching, and maintenance

Auto-scaling - both cluster-level and pod-level scaling

High availability - multi-zone and multi-region clusters available

Integrated with GCP services - Cloud Load Balancing, Cloud Storage, Cloud SQL, etc.

Advanced security features - Network Policy, Pod Security Policy, workload identity

GKE Architecture

A GKE cluster consists of:

Control Plane: Managed by Google, handles Kubernetes API, resource management, and scheduling

Worker Nodes: Compute Engine VMs that run your containers and pods

Networking: VPC networks, Load Balancers, and Cloud DNS

2.3 Lab 1: Creating and Managing GKE Clusters

Estimated Duration: 15-20 minutes

Lab Overview

In this lab, you'll create your first GKE cluster, configure kubectl to access it, and explore the cluster resources. This hands-on experience will familiarize you with cluster management fundamentals.

Lab Objectives

- Create a GKE cluster using gcloud CLI
- Configure kubectl authentication
- Explore cluster resources using kubectl

Step 1: Set Up Your GCP Project

First, set your project ID as an environment variable:

```
export PROJECT_ID=your-project-id
gcloud config set project $PROJECT_ID
```

Step 2: Create a GKE Cluster

Create a cluster named 'gke-tutorial-cluster' in the us-central1 region:

```
gcloud container clusters create gke-tutorial-cluster \
  --zone us-central1-a \
  --num-nodes 3 \
  --machine-type n1-standard-1 \
  --enable-autoscaling \
  --min-nodes 1 \
  --max-nodes 5
```

This command creates a cluster with:

- 3 initial nodes of machine type n1-standard-1
- Auto-scaling enabled between 1 and 5 nodes
- Located in us-central1-a zone



Note: Cluster creation typically takes 5-10 minutes. You'll see status updates in the console.

Step 3: Configure kubectl Authentication

Once the cluster is created, configure kubectl to use your cluster:

```
gcloud container clusters get-credentials gke-tutorial-cluster --zone us-central1-a
```

Verify the connection:

```
kubectl cluster-info
```

Step 4: Explore Your Cluster

View all nodes in your cluster:

```
kubectl get nodes
```

Expected output will show 3 nodes with STATUS 'Ready'.

View namespaces:

```
kubectl get namespaces
```

Check system pods:

```
kubectl get pods --namespace kube-system
```

2.4 Lab 2: Deploying Applications

Estimated Duration: 20-25 minutes

Lab Overview

In this lab, you'll deploy a simple containerized application to your GKE cluster. You'll create a Deployment manifest, deploy it, and expose it via a Service.

Step 1: Create a Sample Application Deployment

Create a file named 'hello-app-deployment.yaml':

```
apiVersion: apps/v1 kind: Deployment metadata: name: hello-app namespace: default spec: replicas: 3
selector: matchLabels: app: hello-app template: metadata: labels: app: hello-app spec:
containers: - name: hello-container image: gcr.io/google-samples/hello-app:1.0 ports: -
containerPort: 8080 resources: requests: cpu: 100m memory: 128Mi limits:
cpu: 500m memory: 512Mi
```

Deployment Manifest Explanation

apiVersion: apps/v1 - Uses the apps API version

kind: Deployment - This is a Deployment resource

metadata.name: hello-app - The deployment name

spec.replicas: 3 - Maintain 3 running pods at all times

image: gcr.io/google-samples/hello-app:1.0 - Pre-built container image

resources - CPU and memory requests and limits for resource management

Step 2: Deploy the Application

Apply the deployment:

```
kubectl apply -f hello-app-deployment.yaml
```

Verify the deployment:

```
kubectl get deployments
```

Check the pods:

```
kubectl get pods -l app=hello-app
```

Step 3: Create a Service to Expose the Application

Create 'hello-app-service.yaml':

```
apiVersion: v1 kind: Service metadata: name: hello-app-service spec: type: LoadBalancer ports: - port: 80
targetPort: 8080 selector: app: hello-app
```

Deploy the service:

```
kubectl apply -f hello-app-service.yaml
```

Step 4: Access Your Application

Get the external IP address:

```
kubectl get service hello-app-service
```

Once the EXTERNAL-IP is assigned (may take a minute), access the application:

```
curl http://EXTERNAL-IP
```

You should see the 'Hello, World!' response with the hostname of the pod that served the request.

2.5 Lab 3: Managing Deployments

Estimated Duration: 25-30 minutes

Lab Overview

This lab covers advanced deployment management including scaling, rolling updates, and rollback operations.

Task 1: Manual Scaling

Scale the deployment to 5 replicas:

```
kubectl scale deployment hello-app --replicas=5
```

Verify the scaling:

```
kubectl get pods -l app=hello-app
```

Watch the pods being created in real-time:

```
kubectl get pods -l app=hello-app -w
```

Task 2: Rolling Update

Update the application image to a new version:

kubectl set image deployment/hello-app hello-container=gcr.io/google-samples/hello-app:2.0
Watch the rollout progress:

kubectl rollout status deployment/hello-app
Check the rollout history:

kubectl rollout history deployment/hello-app

Task 3: Rollback a Deployment

If you need to revert to the previous version:

kubectl rollout undo deployment/hello-app
Verify the rollback:

kubectl get deployment hello-app -o yaml | grep image

Task 4: Autoscaling with Horizontal Pod Autoscaler (HPA)

Create an HPA configuration file 'hello-app-hpa.yaml':

```
apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler metadata: name: hello-app-hpa spec:
scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: hello-app minReplicas: 2 maxReplicas:
10 metrics: - type: Resource resource: name: cpu target: type: Utilization
averageUtilization: 70 - type: Resource resource: name: memory target: type: Utilization
averageUtilization: 80
```

Apply the HPA:

kubectl apply -f hello-app-hpa.yaml
Monitor the HPA status:

kubectl get hpa hello-app-hpa -w

Task 5: Debugging and Logs

View logs from a specific pod:

kubectl logs <pod-name>

View logs from all pods in the deployment:

kubectl logs -l app=hello-app

Describe a pod for detailed information:

kubectl describe pod <pod-name>

Execute a command inside a running pod:

kubectl exec -it <pod-name> -- /bin/sh

3. Cloud Monitoring: Qwik Start

3.1 Cloud Monitoring Overview

What is Cloud Monitoring?

Google Cloud Monitoring (part of Google Cloud's operations suite) provides comprehensive visibility into the performance, uptime, and health of your applications and infrastructure. It automatically collects metrics from your GCP resources and applications, enabling you to:

Monitor resource utilization and performance

Create custom metrics from your applications

Build dashboards for visualization

Set up alerting policies and notifications

Integrate with third-party tools and custom applications

Key Components of Cloud Monitoring

Metrics: Measurements of different aspects of your system. Examples include CPU utilization, memory usage, network traffic, and application-specific metrics.

Time Series: A sequence of metric data points over time. Each data point is associated with a timestamp and label values.

Dashboards: Visualizations of your metrics using charts, graphs, and other visual elements. Dashboards help you understand system health at a glance.

Alerting Policies: Rules that trigger notifications when metrics meet certain conditions. Alerts help you respond to issues promptly.

Notification Channels: Destinations for alert notifications (email, SMS, Slack, PagerDuty, webhooks, etc.)

Monitoring Your GKE Cluster

Cloud Monitoring automatically collects metrics from GKE clusters, including:

Node-level metrics: CPU, memory, disk utilization

Pod-level metrics: CPU, memory usage per pod

Container metrics: Per-container resource usage

Network metrics: Pod-to-pod and external network traffic

3.2 Lab 4: Setting Up Cloud Monitoring

Estimated Duration: 15-20 minutes

Lab Overview

In this lab, you'll enable Cloud Monitoring for your GKE cluster and verify that metrics are being collected.

Step 1: Enable Cloud Monitoring API

Enable the Monitoring API using gcloud:

```
gcloud services enable monitoring.googleapis.com
```

Enable the Logging API as well (often used together with Monitoring):

```
gcloud services enable logging.googleapis.com
```

Step 2: Verify Metrics Collection

Navigate to the Cloud Console and open Monitoring. Go to Metrics Explorer.

Select 'Kubernetes Container' resource type

Choose a metric such as 'container/cpu/core_usage_time'

Filter by your cluster 'gke-tutorial-cluster'

You should see metric data being collected from your running pods.

Step 3: Install Cloud Monitoring Agent (Optional)

For Compute Engine VMs (not needed for GKE as it auto-installs), install the Monitoring agent. For GKE, metrics are collected automatically through the managed service.

Step 4: Explore Available Metrics

In the Cloud Console, go to Monitoring > Metrics Explorer

Explore different resource types:

kubernetes.io/container - Container-level metrics

kubernetes.io/pod - Pod-level metrics

kubernetes.io/node - Node-level metrics

Try visualizing different metrics like CPU usage, memory usage, and network traffic.

3.3 Lab 5: Creating Custom Dashboards

Estimated Duration: 20-25 minutes

Lab Overview

Dashboards provide at-a-glance visibility into your system's health. In this lab, you'll create a custom dashboard for your GKE cluster and hello-app deployment.

Step 1: Create a New Dashboard

In Cloud Console, go to Monitoring > Dashboards

Click 'Create Dashboard'

Name it 'GKE Cluster Health Dashboard'

Step 2: Add Charts to Your Dashboard

Chart 1: Node CPU Utilization

Click 'Add Chart'

Select metric: `kubernetes.io/node/cpu/allocatable_utilization`

Set filter: `resource.cluster_name = 'gke-tutorial-cluster'`

Aggregation: Mean

Chart 2: Pod Count by Deployment

Click 'Add Chart'

Select metric: `kubernetes.io/pod/volume/utilization`

Group by: `resource.labels.pod_name`

Chart 3: Container Memory Usage

Click 'Add Chart'

Select metric: `kubernetes.io/container/memory/used_bytes`

Filter by deployment: `resource.labels.pod_name matches regex '.*hello-app.*'`

Aggregation: Mean

Step 3: Customize Chart Display

For each chart, you can customize:

Chart type (line, area, bar, etc.)

Time range and refresh interval

Y-axis scaling

Color coding and thresholds

Step 4: Save Your Dashboard

Click 'Save' to persist your dashboard

You can now return to this dashboard at any time from the Dashboards list

Step 5: Set Dashboard Refresh and Time Window

At the top of the dashboard, set the time range (Last 1 hour, Last 24 hours, etc.)

Set auto-refresh (every 1 minute, 5 minutes, etc.) to keep data current

3.4 Lab 6: Alerts and Notification Policies

Estimated Duration: 20-25 minutes

Lab Overview

Alerting policies notify you when your systems deviate from desired behavior. In this lab, you'll create alert policies for CPU and memory utilization.

Step 1: Create a Notification Channel

Before creating alert policies, set up a notification channel for receiving alerts.

In Cloud Console, go to Monitoring > Notification Channels

Click 'Create Channel'

Select 'Email' as the notification type

Enter your email address

Click 'Create Channel'

Verify your email by clicking the confirmation link sent to your inbox

Step 2: Create an Alert for High CPU Usage

In Cloud Console, go to Monitoring > Alert Policies

Click 'Create Policy'

Fill in the alert condition:

Display Name: 'High CPU Usage on GKE Nodes'

Metric: `kubernetes.io/node/cpu/allocatable_utilization`

Condition: is above 80%

Duration: For at least 2 minutes

Add the notification channel you created earlier

Click 'Create Policy'

Step 3: Create an Alert for High Memory Usage

Repeat the process with these settings:

Display Name: 'High Memory Usage on GKE Nodes'

Metric: `kubernetes.io/node/memory/allocatable_utilization`

Condition: is above 85%

Duration: For at least 2 minutes

Step 4: Create an Alert for Pod Restart

Create another alert policy:

Display Name: 'High Pod Restart Rate'

Metric: `kubernetes.io/container/restart_count`

Condition: Increases by more than 2 in 5 minutes

Step 5: Test Your Alert (Optional)

To test if alerts are working, you can intentionally create high CPU usage. In a real scenario, you might run a load test:

```
# Deploy a load generation pod
kubectl run load-generator --image=busybox -- /bin/sh -c 'while true; do echo "load"; done'
```

Monitor your email for alert notifications

Clean up by deleting the load generator:

```
kubectl delete pod load-generator
```

4. Advanced Topics

4.1 Custom Metrics and Application Monitoring

While Cloud Monitoring provides infrastructure metrics automatically, you can also send custom metrics from your applications. This is useful for application-specific KPIs.

Example: Publishing Custom Metrics from a Python Application

```
from google.cloud import monitoring_v3
import time
client = monitoring_v3.MetricServiceClient()
project_name = f"projects/YOUR_PROJECT_ID"
series = monitoring_v3.TimeSeries()
series.metric.type = 'custom.googleapis.com/my_metric'
series.resource.type = 'global'
now = time.time()
seconds = int(now)
nanos = int((now - seconds) * 10 ** 9)
interval = monitoring_v3.TimeInterval({"end_time": {"seconds": seconds,
```

```
"nanos": nanos}}) point = monitoring_v3.Point({"interval": interval, "value": {"double_value": 42.0}})
series.points = [point] client.create_time_series(name=project_name, time_series=[series])
```

4.2 Multi-Cluster Monitoring

When running multiple GKE clusters, you can create dashboards and alerts that span all clusters:

Use cluster_name label in metrics to distinguish clusters

Create 'uber-dashboards' combining metrics from multiple clusters

Set up centralized alerting across all clusters

4.3 Service Level Indicators (SLIs) and Error Budgets

As you mature your monitoring practice, consider implementing SLIs:

Define SLOs (Service Level Objectives) for your services

Track error budgets to understand how much downtime you can afford

Use Cloud Monitoring to measure SLI compliance

4.4 Integration with Third-Party Tools

Cloud Monitoring integrates with:

Datadog, New Relic, Dynatrace - via Cloud Monitoring API

Slack, PagerDuty, OpsGenie - via notification channels

Cloud Logging - for centralized log analysis

Terraform - for infrastructure-as-code monitoring configuration

5. Troubleshooting Guide

Common Issues and Solutions

Issue 1: Pods stuck in 'Pending' state

Symptoms: Pods show 'Pending' status for extended period

Debugging steps:

kubectl describe pod <pod-name>

Check 'Events' section for reasons

kubectl get nodes - Verify nodes are Ready

Solutions:

Insufficient resources: Scale cluster or adjust resource requests

Node selectors: Verify node labels match pod requirements

Taints and tolerations: Check if nodes have taints blocking pod scheduling

Issue 2: High latency or slow response times

Investigation steps:

Check network policies restricting traffic

Monitor CPU and memory from Cloud Monitoring dashboards

Check logs for application errors:

kubectl logs -f <pod-name>

Verify database connections and external API calls

Issue 3: Metrics not showing in Cloud Monitoring

Verify Cloud Monitoring API is enabled:

gcloud services list --enabled | grep monitoring

Check cluster has necessary IAM permissions

Wait 2-3 minutes for initial metrics to appear (there's a collection delay)

Verify monitoring agent is running on nodes:

```
kubectrl get daemonset -A | grep monitoring
```

Issue 4: Alert policies not triggering

Verification steps:

Verify notification channel is verified/enabled

Check alert condition threshold - verify metric actually exceeds threshold

Review policy condition duration - may need longer evaluation window

Check notification channel in spam folder if using email

6. Best Practices and Summary

Kubernetes Best Practices

1. Resource Requests and Limits

Always define resource requests and limits for your containers. This ensures proper scheduling and prevents resource contention.

2. Use Namespaces for Isolation

Organize your applications into namespaces for logical separation and easier management.

3. Implement Health Checks

Define liveness and readiness probes to ensure Kubernetes can detect and replace unhealthy pods.

4. Use ConfigMaps and Secrets

Separate configuration from code using ConfigMaps and Secrets, making your deployments more flexible and secure.

5. Implement Network Policies

Use Network Policies to control traffic between pods and external networks, enhancing security.

Monitoring Best Practices

1. Monitor Business Metrics

Don't just monitor infrastructure. Track application-level metrics that matter to your business.

2. Set Meaningful Thresholds

Alert thresholds should be based on actual impact to users, not arbitrary numbers.

3. Use Dashboards Strategically

Create role-specific dashboards: executive dashboards for high-level health, engineering dashboards for detailed troubleshooting.

4. Correlate Metrics and Logs

Use Cloud Logging alongside Cloud Monitoring to correlate metric spikes with log entries.

5. Plan for Alert Fatigue

Too many alerts lead to alert fatigue. Be selective and make sure every alert is actionable.

Summary

Throughout this tutorial, you've learned:

Kubernetes fundamentals and how Google Kubernetes Engine simplifies cluster management

How to create clusters, deploy applications, and manage deployments with rolling updates and scaling

Cloud Monitoring capabilities for comprehensive observability

Creating dashboards and setting up alerting for proactive issue detection

These skills form the foundation for running production-grade containerized applications on Google Cloud Platform. Continue to explore advanced topics like service meshes, multi-cluster deployments, and advanced security configurations as you deepen your expertise.

Next Steps

Experiment with different deployment strategies (canary deployments, blue-green deployments)

Implement service mesh (Istio) for advanced traffic management

Set up CI/CD pipelines using Cloud Build for automated deployments

Explore Cloud SQL and Cloud Storage integration with your applications

Implement centralized logging with Cloud Logging for comprehensive observability